

Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores
Introducción a la Programación

Documentación Técnica Crazy Snack Rush TEC

Estudiantes:

Jehudy Sanchez Gomez

Alexander Rojas Lai

Profesor:

Leonardo Araya Martinez

Semestre I, 2026

1. Introducción

1.1. Propósito del documento

El presente documento describe la arquitectura, diseño e implementación del sistema Crazy Snack Rush TEC, destinado al profesor y evaluadores del curso de Introducción a la Programación del Instituto Tecnológico de Costa Rica. Este documento sirve como referencia técnica que detalla las decisiones de diseño, la estructura del código y las funcionalidades implementadas.

1.2. Descripción del proyecto

Crazy Snack Rush TEC es un videojuego didáctico inspirado en la dinámica del juego Overcooked 2, adaptado al contexto académico del TEC. Es una aplicación de juego en tiempo real con arquitectura cliente-servidor: el frontend está desarrollado en Unreal Engine 5.7 y el backend en Python con FastAPI. El juego permite a un jugador controlar dos chefs en una cocina, preparando y entregando recetas dentro de un tiempo límite. El usuario final previsto es el estudiante universitario que busca aprender conceptos de programación como los relacionados a la programación orientada a objetos y la comunicación cliente-servidor.

1.3. Objetivos y alcance

Objetivo principal: Desarrollar una versión simplificada de Overcooked 2 aplicando los principios de programación orientada a objetos en Python.

Objetivos específicos:

- Implementar un sistema de clases con herencia y polimorfismo para modelar los elementos del juego.
- Diseñar y exponer endpoints REST que permitan al frontend interactuar con la lógica del juego.
- Implementar un sistema de recetas cargadas desde archivos JSON con validación de ingredientes.
- Implementar un sistema de puntuación y temporizador con penalizaciones por órdenes vencidas.

Alcance del proyecto:

- El juego tiene varios escenarios, cada uno con recetas distintas.
- Un jugador que controla dos chefs
- El sistema no incluye base de datos persistente; el estado se mantiene en memoria.

1.4. Definiciones, acrónimos y abreviaturas

Término	Definición
API REST	Interfaz de programación basada en HTTP que expone endpoints para la comunicación cliente-servidor.
FastAPI	Framework de Python para construir APIs REST
Pydantic	Librería de Python para validación y serialización de datos mediante modelos tipados.
DTO	Data Transfer Object: objeto que transporta datos entre el Unreal y Python.
GameState	Objeto en memoria que mantiene el estado del juego en el servidor.
asyncio.Lock	Mecanismo de bloqueo asíncrono para evitar condiciones de carrera en el GameState.
JSON	JavaScript Object Notation: formato de texto para almacenar datos estructurados.
UML	Unified Modeling Language: lenguaje de modelado para describir arquitecturas de software.

2. Descripción General del Sistema

2.1. Perspectiva del producto

Crazy Snack Rush TEC es un sistema de dos componentes que se comunican mediante HTTP. El frontend en Unreal Engine 5.7 maneja la visualización, movimiento de personajes y captura de entradas del teclado. El backend en Python/FastAPI procesa la lógica del juego: estado de ingredientes, gestión de órdenes, validación de recetas y cálculo de puntuación. El sistema no depende de servicios externos ni bases de datos; los datos de ingredientes y recetas se cargan desde archivos JSON locales al iniciar el servidor.

2.2. Funcionalidades principales

- Gestión de cocina: control de las estaciones, generación aleatoria de órdenes y control del temporizador.
- Sistema de ingredientes: definición de propiedades (canCut, canFry, canBoil, canDeepFry) cargadas desde JSON, con herencia de clases para vegetales, proteínas y bases.
- Sistema de recetas: carga desde JSON con validación de ingredientes requeridos y sus estados.

- Interacción con estaciones: tabla de picar, cocina, freidora, batidora, pila, basura y estación de entrega, cada una con lógica propia.
- Sistema de puntuación: suma de puntos al entregar recetas correctamente, con penalizaciones por órdenes vencidas.
- Concurrencia segura: uso de `async.Lock` para proteger el estado compartido ante múltiples requests simultáneos.

2.3. Restricciones, supuestos y dependencias

Restricciones:

- El backend debe responder rápidamente para no afectar la experiencia de juego.
- El estado del juego se mantiene en memoria; si el servidor se reinicia, la partida se pierde.

Supuestos:

- Unreal Engine y el servidor FastAPI corren en la misma red local.
- Los archivos `ingredients.json` y `recipes.json` están disponibles en la carpeta `components/` al iniciar el servidor.
- El frontend es responsable de controlar el tiempo real entre llamadas a `/game/tick`.

Dependencias:

- Python 3.12+ — lenguaje principal del backend.
- FastAPI 0.136.3 — framework para exposición de endpoints REST.
- Pydantic 2.13.4 — validación y serialización de modelos de datos.
- Unreal Engine 5.7 — motor de videojuegos para el frontend.
- `asyncio` — librería estándar de Python para programación asíncrona.

3. Requisitos del Sistema

3.1. Requisitos funcionales

ID	Descripción	Prioridad	Criterio de Aceptación
RF-001	El sistema debe iniciar la cocina con las estaciones de trabajo enviadas por Unreal.	Alta	El endpoint /game/start responde con bSuccess=true y registra los muebles en el GameState.
RF-002	El chef debe poder interactuar con la despensa para obtener ingredientes.	Alta	Al interactuar con IngredientBox vacío, el chef recibe el ingrediente correspondiente.
RF-003	Los ingredientes deben poder ser procesados en las estaciones correspondientes según sus propiedades.	Alta	La tabla de picar solo corta ingredientes con canCut=true; la cocina solo fríe con canFry=true, etc.
RF-004	El sistema debe generar órdenes aleatorias automáticamente mientras hay tiempo.	Alta	generate_random_order añade órdenes cuando hay menos de 5 activas y time_remaining > 40s.
RF-005	El sistema debe validar y completar una orden al entregarla en la estación de entrega.	Alta	Si los ingredientes entregados corresponden a una orden activa con canDeliver=true, se suman los puntos.
RF-006	Las órdenes deben tener un temporizador y penalizarse si se vencen.	Media	Cada tick reduce timeRemaining; al llegar a 0 los puntos se reducen a la mitad.
RF-007	El chef debe poder desechar ingredientes en la basura.	Media	Al interactuar con trash, el held del chef queda vacío (IngredientDto sin campos).

3.2. Requisitos no funcionales

ID	Categoría	Descripción	Métrica / Valor Esperado
RNF-001	Rendimiento	El backend debe responder a cada request en tiempo aceptable para no afectar la experiencia de juego.	Tiempo de respuesta < 500ms por endpoint.
RNF-002	Concurrencia	El sistema debe manejar múltiples requests sin corromper el estado del juego.	Uso de asyncio.Lock en todos los endpoints que modifican el GameState.
RNF-003	Mantenibilidad	El código debe seguir los principios de POO con herencia y polimorfismo.	Todas las clases de ingredientes heredan de Ingredient; todos los muebles heredan de Furniture.
RNF-004	Extensibilidad	El sistema de ingredientes y recetas debe ser configurable sin modificar código.	Nuevos ingredientes y recetas se agregan únicamente en los archivos JSON.

4. Diseño de la Solución

4.1. Visión general de la arquitectura

El sistema sigue una arquitectura cliente-servidor. El cliente es el frontend en Unreal Engine 5.7, responsable de la visualización, el movimiento de personajes y la captura de input del jugador. El servidor es el backend en Python/FastAPI, responsable de toda la lógica del juego: procesamiento de interacciones, validación de recetas, gestión de órdenes y cálculo de puntuación. Ambos componentes se comunican mediante HTTP usando el patrón REST: Unreal envía requests POST con datos del juego en formato JSON y el backend responde con el estado actualizado.

Esta separación fue elegida porque permite que cada integrante trabaje de manera independiente en su parte con mayor afinidad: el integrante que trabaja el frontend en C++/Unreal y el que trabaja en el backend en Python.

4.2. Descripción de componentes

Componente	Responsabilidad	Tecnología	Dependencias
Frontend (Unreal)	Visualización, movimiento de chefs, captura de input y llamadas a la API.	Unreal Engine 5.7, C++	API REST del Backend
Backend / API	Lógica del juego, validación de ingredientes, gestión de órdenes y puntuación.	Python 3.12, FastAPI 0.136	Archivos JSON locales
GameState	Objeto en memoria que mantiene el estado global del juego durante una partida.	Python (asyncio.Lock)	Kitchen, Furniture
Archivos JSON	Almacenamiento de ingredientes disponibles y recetas del juego.	JSON	Ninguna

4.3. Stack tecnológico

Categoría	Tecnología	Versión	Propósito
-----------	------------	---------	-----------

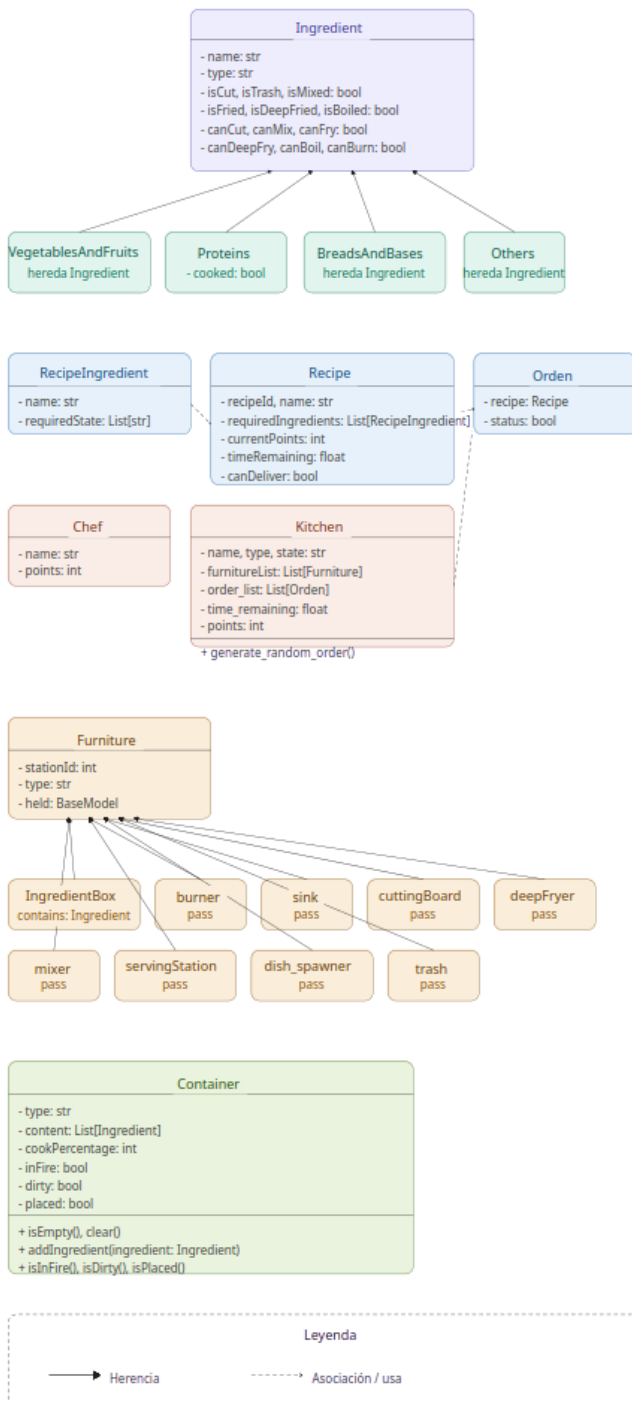
Categoría	Tecnología	Versión	Propósito
Lenguaje Backend	Python	3.12	Lógica del juego
Framework Backend	FastAPI	0.136.3	Exposición de endpoints REST
Validación de datos	Pydantic	2.13.4	Modelado y validación de DTOs
Motor de juego	Unreal Engine	5.7	Frontend: visualización y control
Concurrencia	asyncio	Stdlib Python	Manejo de requests simultáneos
Control de versiones	Git / GitHub	—	Gestión del código fuente

4.4. Diagrama de clases y modelo de datos

El diagrama de clases muestra la jerarquía de clases del backend. Las clases principales son:

- Ingredient (clase base): define los atributos comunes de todos los ingredientes (estados is*, propiedades can*). Sus subclasses son VegetablesAndFruits, Proteins (con atributo adicional cooked), BreadsAndBases y Others.
- Recipe y RecipeIngredient: modelan las recetas y sus ingredientes requeridos con estados específicos. Se cargan desde recipes.json.
- Order: representa una orden activa del juego, compuesta por una Recipe y un estado (isDone).
- Kitchen: clase central del GameState que contiene la lista de muebles, órdenes activas, tiempo restante y puntaje. Incluye el método generate_random_order.
- Furniture (clase base): representa un mueble del juego con stationId, type y held. Sus subclasses implementan comportamientos específicos por estación: IngredientBox, burner, cuttingBoard, mixer, deepFryer, trash, servingStation, sink, dish_spawner, table.
- Container: modela contenedores físicos (ollas, platos, tazas) con métodos isEmpty, clear, addIngredient, isInFire, isDirty, isPlaced.
- Chef: modela un chef con nombre y puntos acumulados.

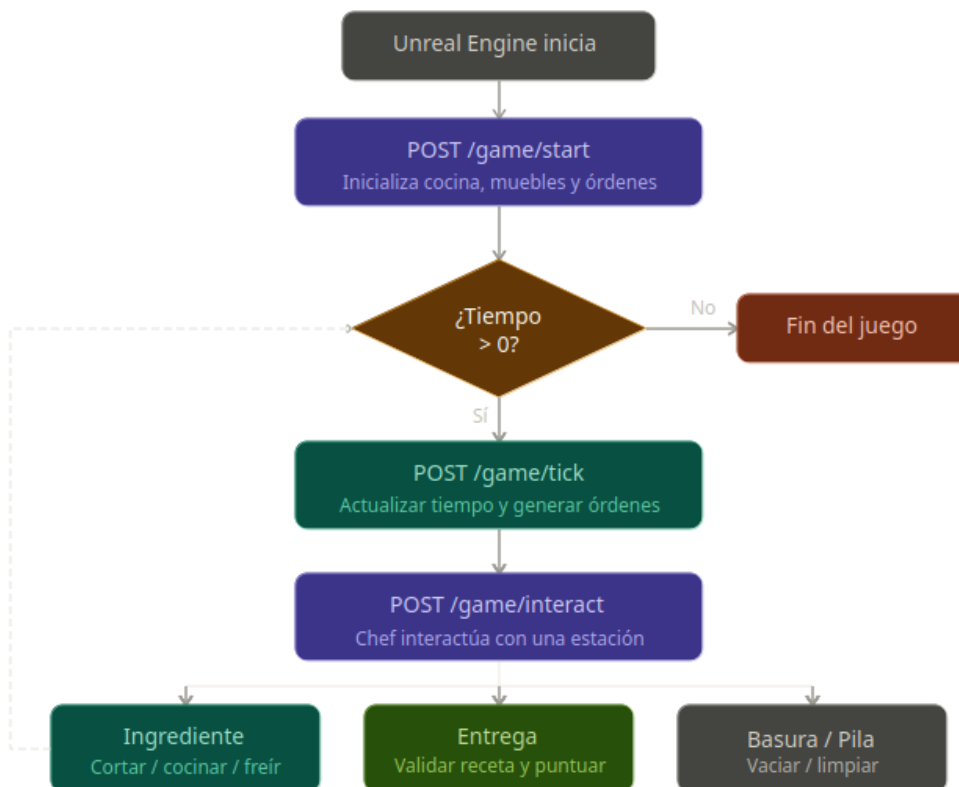
El modelo de datos no utiliza base de datos relacional. Los ingredientes y recetas se persisten en archivos JSON (ingredients.json y recipes.json) y se cargan en memoria al iniciar el servidor. El estado del juego vive únicamente en el objeto GameState mientras el servidor está corriendo.



4.5. Diagrama de flujo y secuencia

El flujo principal del juego sigue esta secuencia:

1. Unreal Engine inicia el juego y llama a POST /game/start enviando la lista de muebles del mapa.
2. El backend inicializa el GameState, registra los muebles y genera las primeras órdenes aleatorias.
3. En cada frame, Unreal llama a POST /game/tick para actualizar el tiempo restante y las órdenes.
4. Cuando el jugador interactúa con una estación, Unreal llama a POST /game/interact con el chefId, stationId, action y el ingrediente que lleva en la mano.
5. El backend busca la estación, verifica las propiedades del ingrediente usando find_ingredient(), aplica la transformación correspondiente y retorna el ingrediente actualizado.
6. Al entregar en servingStation, el backend valida que la receta entregada coincida con una orden activa, suma los puntos y retorna el puntaje actualizado.



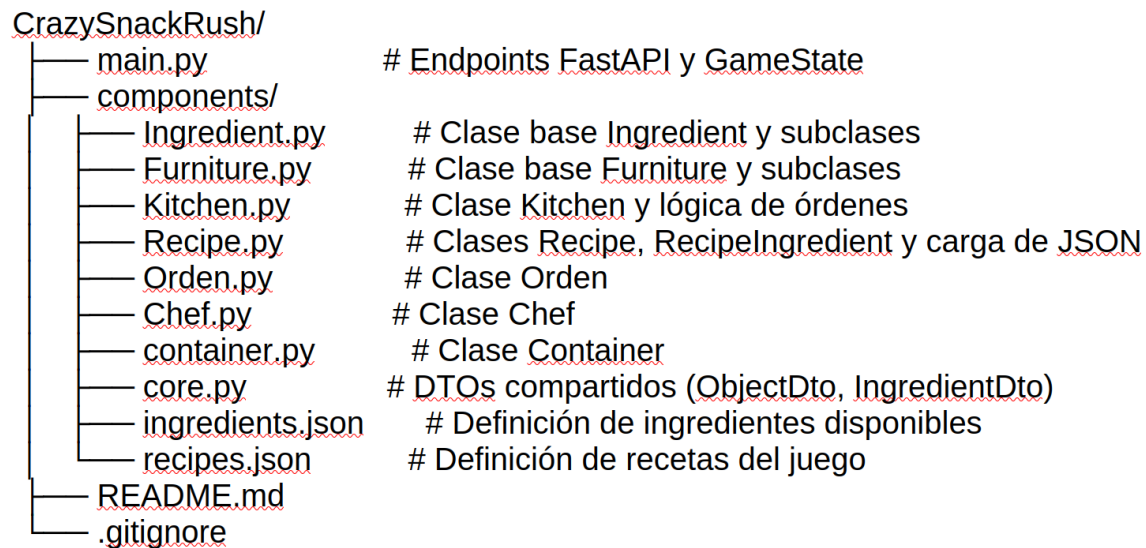
4.6. Decisiones de diseño y justificación

Decisión	Alternativas Evaluadas	Opción Elegida	Justificación
Gestión del estado del juego	Estado en Unreal (stateless backend), estado en base de datos, estado en memoria del servidor.	Estado en memoria del servidor (GameState)	Simplicidad de implementación y latencia mínima. No requiere base de datos para un proyecto académico con sesión única.
Arquitectura	Monolítica (todo en Unreal), microservicios, cliente-servidor.	Cliente-servidor (Unreal + FastAPI)	Permite separación de responsabilidades entre el equipo de frontend y backend, con un contrato claro de API.
Persistencia de ingredientes y recetas	Hardcodeados en Python, base de datos, archivos JSON.	Archivos JSON	Permiten agregar o modificar ingredientes y recetas sin tocar el código, cumpliendo el requisito de extensibilidad.
Concurrencia	Ignorarla (single-threaded), threading, asyncio.	asyncio.Lock por GameState	FastAPI es asíncrono por naturaleza; asyncio.Lock es la solución idiomática para proteger recursos compartidos sin overhead de threads.
Herencia de ingredientes	Un solo modelo plano, herencia, composición.	Herencia (VegetablesAndFruits, Proteins, BreadsAndBases, Others)	El proyecto requiere demostrar POO con herencia. Proteins tiene comportamiento adicional (cooked) que justifica la subclase.

5. Implementación y Validación de Calidad

5.1. Estructura del repositorio

El repositorio del proyecto en GitHub (Arcrein/CrazySnackRush) está organizado de la siguiente manera:



5.2. Entorno de desarrollo, guía de instalación y ejecución

Entorno:

- Sistema operativo: Ubuntu 24.04 (desarrollo del backend) / Windows 11 (desarrollo del frontend)
- Lenguaje: Python 3.12+
- Motor de videojuegos: Unreal Engine 5.7

Guía de instalación y ejecución del backend:

1. Clonar el repositorio

```
git clone https://github.com/Liaaaaaa28/CrazySnackRush.git
cd CrazySnackRush
```

2. Crear entorno virtual

```
python3 -m venv venv
source venv/bin/activate # Linux/Mac
# venv\Scripts\activate # Windows
```

3. Instalar dependencias

```
pip install fastapi pydantic uvicorn
```

4. Ejecutar el servidor

```
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

5.3. Estrategias y casos de prueba

Estrategia de pruebas:

Tipo	Descripción	Herramienta	Cobertura
Pruebas manuales de API	Verificación de endpoints mediante requests HTTP directos.	FastAPI /docs (Swagger UI)	Endpoints principales
Pruebas de integración	Verificación del flujo completo desde Unreal hasta el backend.	Unreal Engine + servidor local	Flujos críticos del juego

Casos de prueba:

ID	Descripción	Pasos	Resultado Esperado	Resultado Obtenido	Estado
CP-001	Iniciar cocina	POST /game/start con lista de muebles válida.	bSuccess=true, muebles registrados en GameState.	bSuccess=true	
CP-002	Cortar vegetal válido	POST /game/interact con stationId=cutting_board y tomate crudo.	Ingrediente retorna con state=Cortado.	state=Cortado	
CP-003	Cortar ingrediente no cortable	POST /game/interact con stationId=cutting_board y huevo.	bSuccess=false.	bSuccess=false	
CP-004	Freír en freidora	POST /game/interact con stationId=deepFryer y chocolate batter.	state=DeepFried.	state=DeepFried	
CP-005	Desechar ingrediente	POST /game/interact con stationId=trash.	UpdatedHeldIngredient vacío.	Ingredient Dto vacío	

ID	Descripción	Pasos	Resultado Esperado	Resultado Obtenido	Estado
CP-006	Limpiar plato sucio	POST /game/interact con stationId=sink y Plate/Dirty.	state=Clean.	state=Clean	
CP-007	Generación de órdenes	POST /game/tick con time_remaining=120.	Nueva orden generada si hay < 5 órdenes activas.	Orden añadida	

5.4. Resultados de pruebas y problemas encontrados

Resultados:

- Todos los endpoints de interacción con estaciones responden correctamente para los casos válidos e inválidos.
- El sistema de generación de órdenes funciona correctamente con el temporizador.
- El asyncio.Lock protege efectivamente el GameState ante requests concurrentes.

Problemas solucionados:

- Error externally-managed-environment al instalar paquetes en Ubuntu: solucionado creando un entorno virtual con python3 -m venv.
- Conflicto de nombres entre el parámetro request de InteractRequest y el parámetro Request de FastAPI: solucionado renombrando el primero a data.
- Pérdida de cambios locales al hacer git pull: solucionado usando git stash antes de pull.

Problemas no solucionados:

6. Declaración de Uso de Inteligencia Artificial

Principio de Transparencia Académica

El uso de herramientas de Inteligencia Artificial en este proyecto se declara de forma explícita y responsable, siguiendo los principios de integridad académica del TEC.

6.1. Herramientas de IA utilizadas

Herramienta de IA	Tipo	Período de uso	Propósito
Claude (Anthropic)	Modelo de lenguaje (LLM)	Mayo-Junio 2026	Aprendizaje y preguntas acerca del código y solución de problemas
Codex (OpenAI)	Modelo de lenguaje (LLM)	Mayo-Junio 2026	Aprendizaje y preguntas acerca del código.

6.2. Descripción del uso de inteligencia artificial

Áreas de uso:

- Generación y asistencia en código: estructura de clases con Pydantic, endpoints FastAPI, lógica de interacción con estaciones, sistema de órdenes y puntuación.
- Redacción y documentación: apoyo en la redacción de esta documentación técnica.
- Investigación y resolución de problemas: depuración de errores de importación, conflictos de git

6.3. Declaración de responsabilidad y autoría

Los suscritos, integrantes del equipo de trabajo del proyecto "Crazy Snack Rush TEC", declaramos que:

1. El diseño, análisis, desarrollo y conclusiones del presente proyecto son resultado de nuestro trabajo intelectual y esfuerzo como equipo.
2. Las herramientas de Inteligencia Artificial fueron utilizadas exclusivamente como apoyo en tareas específicas descritas en este capítulo, no como sustituto de nuestro trabajo.
3. Todo el contenido generado por IA fue revisado, comprendido, validado y, cuando fue necesario, corregido por los integrantes del equipo
4. Asumimos plena responsabilidad académica sobre el contenido total de este proyecto y su documentación.

5. El uso de IA se realizó conforme a las políticas de la institución educativa y los lineamientos del docente responsable.

Firma de los integrantes del grupo:

J Jehudy Sánchez Gómez

Alexander Pengyang Rojas Lai

7. Conclusión

El desarrollo de Crazy Snack Rush TEC permitió al equipo aplicar de manera práctica los conceptos fundamentales de la programación orientada a objetos: herencia, encapsulamiento y polimorfismo. La jerarquía de clases diseñada, con Ingredient y Furniture como clases base y sus respectivas subclases, demostró cómo la POO permite modelar de forma natural un dominio complejo como el de un videojuego de cocina.

La arquitectura cliente-servidor resultó ser una decisión acertada para el contexto del proyecto, ya que permitió que el integrante que trabajó el frontend (Unreal Engine) y el que trabajó en el backend (Python/FastAPI) trabajaran de forma independiente. El uso de Pydantic para los DTOs garantizó que los datos intercambiados entre ambos sistemas fueran siempre válidos y con tipos correctos, reduciendo errores de integración.

Uno de los principales retos técnicos fue la gestión del estado del juego en un entorno asíncrono. El uso de `asyncio.Lock` para proteger el `GameState` fue una solución eficaz, aunque planteó la necesidad de entender conceptos avanzados de concurrencia que van más allá del alcance básico del curso.

Como mejoras futuras, se identifica la necesidad de implementar completamente la clase `Mix` para la batidora, agregar pruebas automatizadas con `Pytest`, y resolver el conflicto arquitectónico entre el sistema de interacción legacy (`if/elif` en `main.py`) y el nuevo sistema orientado a objetos (`doInteract` en `Furniture`). También sería valioso agregar persistencia con una base de datos para guardar puntajes históricos.

En términos de aprendizaje, este proyecto demostró que la diferencia entre código que simplemente funciona y código bien diseñado radica en la aplicación disciplinada de principios de POO y en la comunicación constante dentro del equipo de desarrollo.

8. Bibliografía

[1] S. Ramirez, FastAPI Documentation, Tiangolo, 2024. [En línea]. Disponible en: <https://fastapi.tiangolo.com>

[2] Pydantic Team, Pydantic Documentation v2, 2024. [En línea]. Disponible en: <https://docs.pydantic.dev/latest/>

[3] Python Software Foundation, asyncio — Asynchronous I/O, Python 3.12 Documentation, 2024. [En línea]. Disponible en: <https://docs.python.org/3/library/asyncio.html>

[4] Epic Games, Unreal Engine 5.7 Documentation, 2024. [En línea]. Disponible en: <https://dev.epicgames.com/documentation/en-us/unreal-engine/unreal-engine-5-0-documentation>

[5] Ghost Town Games, Overcooked! 2, Team17, 2018. [En línea]. Referencia de diseño del juego: <https://www.youtube.com/watch?v=Fo5CR7utRik>

[6] ISO/IEC, Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE), ISO/IEC 25010:2011, 2011.

[7] Google LLC, Google Cloud Architecture Framework, 2024. [En línea]. Disponible en: <https://cloud.google.com/architecture/framework>